

SQL Practice Queries

Lesson 3 - Single Row Functions and Operators

Prepared for DTank54 Group Students
Oracle HR Schema - A1 English Level

Important Notes Before You Start

In Oracle SQL, single quotes are for string values (like 'IT'). Double quotes are for column aliases that have spaces (like "Department Name"). Be careful with this difference when you write your queries.

The HR schema has 7 tables: REGIONS, COUNTRIES, LOCATIONS, DEPARTMENTS, JOBS, EMPLOYEES, and JOB_HISTORY. All queries in this document use these tables.

Section 1: IN Operator

The IN operator lets you check if a value is in a list. Instead of writing many OR conditions, you can use IN to make your query shorter and easier to read. This is very useful when you want to filter data by a list of specific values.

Query 1 - IN - Basic

```
SELECT first_name, last_name, department_id
FROM employees
WHERE department_id IN (10, 20, 30);
```

What this query does: This query shows the first name, last name, and department ID of employees who work in department 10, 20, or 30. The IN operator checks if department_id is in the list (10, 20, 30). It is the same as writing department_id = 10 OR department_id = 20 OR department_id = 30, but shorter.

Answer: Returns employees from departments 10 (Administration), 20 (Marketing), and 30 (Purchasing).

Query 2 - IN - Character Values

```
SELECT first_name, last_name, job_id
FROM employees
WHERE job_id IN ('IT_PROG', 'SA_REP', 'ST_CLERK');
```

What this query does: This query finds employees whose job ID is IT_PROG, SA_REP, or ST_CLERK. When you use IN with text values, you must put each value in single quotes. This helps you find employees in specific job roles.

Query 3 - IN - NOT IN

```
SELECT first_name, last_name, department_id
FROM employees
WHERE department_id NOT IN (10, 20, 30)
ORDER BY department_id;
```

What this query does: This query shows employees who do NOT work in departments 10, 20, or 30. The NOT IN operator is the opposite of IN. It excludes the values in the list. The ORDER BY clause sorts the results by department_id so the output is easy to read.

Section 2: NOT Operator

The NOT operator reverses a condition. You can use it with many operators like BETWEEN, LIKE, and NULL. It is very helpful when you want to find rows that do not match a condition. You can write NOT before the operator or use NOT IN, NOT LIKE, NOT BETWEEN, and IS NOT NULL.

Query 4 - NOT - With BETWEEN

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary NOT BETWEEN 5000 AND 10000
ORDER BY salary;
```

What this query does: This query shows employees whose salary is NOT between 5000 and 10000. The NOT BETWEEN operator finds salaries that are less than 5000 or greater than 10000. It is the opposite of BETWEEN.

Query 5 - NOT LIKE

```
SELECT first_name, last_name
FROM employees
WHERE first_name NOT LIKE 'S%'
ORDER BY first_name;
```

What this query does: This query shows employees whose first name does NOT start with the letter S. The NOT LIKE operator is the opposite of LIKE. The percent sign (%) means any characters after S, so NOT LIKE 'S%' excludes all names starting with S.

Query 6 - IS NOT NULL

```
SELECT employee_id, first_name, last_name, commission_pct
FROM employees
WHERE commission_pct IS NOT NULL;
```

What this query does: This query shows employees who have a commission percentage. In Oracle, NULL means the value is empty or unknown. IS NOT NULL finds rows where the column has a value (not empty). Only sales representatives have commission_pct in the HR schema.

Section 3: LIKE Operator

The LIKE operator searches for a pattern in text data. You use special symbols to build the pattern: the percent sign (%) means zero or more characters, and the underscore (_) means exactly one character. LIKE is case-sensitive in Oracle.

Query 7 - LIKE - Starts With

```
SELECT first_name, last_name
FROM employees
WHERE first_name LIKE 'S%'
ORDER BY first_name;
```

What this query does: This query finds employees whose first name starts with the letter S. The pattern 'S%' means: the letter S followed by any number of characters (including zero characters). So it matches 'Steven', 'Shelley', 'Shanta', and so on.

Query 8 - LIKE - Contains

```
SELECT first_name, last_name
FROM employees
WHERE last_name LIKE '%an%'
ORDER BY last_name;
```

What this query does: This query finds employees whose last name contains the letters 'an' anywhere in the name. The pattern '%an%' means: any characters, then 'an', then any characters. So it matches names like 'Grant', 'Rogers', 'Banda', etc.

Query 9 - LIKE - Exact Length Pattern

```
SELECT first_name
FROM employees
WHERE first_name LIKE '_____'
ORDER BY first_name;
```

What this query does: This query finds employees whose first name has exactly 5 letters. Each underscore (_) means exactly one character. Five underscores = exactly 5 characters. So it matches names like 'David', 'Bruce', 'Karen', etc.

Query 10 - LIKE - Ends With

```
SELECT first_name, last_name
FROM employees
WHERE last_name LIKE '%son'
ORDER BY last_name;
```

What this query does: This query finds employees whose last name ends with 'son'. The pattern '%son' means: any characters followed by 'son'. So it matches names like 'Johnson', 'Jackson', 'Mason', etc.

Section 4: IS NULL / IS NOT NULL

NULL means a value is missing or unknown. In Oracle, you cannot use = or != to compare with NULL. You must use IS NULL to find empty values and IS NOT NULL to find values that exist. This is a very common operation in data analysis because missing data is a big problem in real databases.

Query 11 - IS NULL - Find Missing Data

```
SELECT employee_id, first_name, last_name, manager_id
FROM employees
WHERE manager_id IS NULL;
```

What this query does: This query finds employees who do not have a manager. In the HR schema, only the top manager (the CEO) has no manager_id. This query is useful to find the highest-level person in the company.

Query 12 - IS NULL - Commission Check

```
SELECT employee_id, first_name, last_name, commission_pct, job_id
FROM employees
WHERE commission_pct IS NULL
ORDER BY employee_id;
```

What this query does: This query shows employees who do not earn a commission. Most employees do not have a commission_pct value (it is NULL). Only sales jobs (like SA_REP) have commission. This query helps you understand which employees do not get extra pay from sales.

Section 5: Concatenation (||)

The double pipe symbol (||) joins two or more text values together into one string. You can connect column values with text. This is very useful when you want to show a full name or create a custom label for a column.

Query 13 - || - Basic Concatenation

```
SELECT first_name || ' ' || last_name AS "Full Name"
FROM employees
WHERE department_id = 10;
```

What this query does: This query joins the first name, a space, and the last name together. The result is a new column called 'Full Name'. For example, if first_name is 'Jennifer' and last_name is 'Whalen', the result is 'Jennifer Whalen'. Note that the alias uses double quotes because it has a space.

Query 14 - || - Concatenation with Labels

```
SELECT first_name || ' works as ' || job_id || ' in department ' ||
department_id
AS "Employee Info"
FROM employees
WHERE department_id = 90;
```

What this query does: This query creates a full sentence for each employee in department 90. It joins text strings like ' works as ' with column values like job_id and department_id. The result looks like: 'Steven works as AD_PRES in department 90'. This is helpful for creating readable reports.

Query 15 - || - Email Address Format

```
SELECT lower(first_name || '.' || last_name || '@company.com') AS "Email"
FROM employees
WHERE department_id = 10;
```

What this query does: This query creates an email address from the employee's first name and last name. It uses LOWER() to make all letters small, and || to join the parts. For example: 'jennifer.whalen@company.com'. The double quotes in the alias are needed because of the space.

Section 6: CASE Expression

The CASE expression is like IF-THEN-ELSE in other programming languages. It lets you create conditional logic in your SQL query. You can show different results based on a condition. CASE is very useful for data analysis because you can group data into categories.

Query 16 - CASE - Salary Categories

```
SELECT first_name, last_name, salary,
CASE
    WHEN salary < 5000 THEN 'Low'
    WHEN salary BETWEEN 5000 AND 10000 THEN 'Medium'
    WHEN salary > 10000 THEN 'High'
END AS "Salary Level"
FROM employees
ORDER BY salary;
```

What this query does: This query puts each employee into a salary category: Low (less than 5000), Medium (5000 to 10000), or High (more than 10000). The CASE expression checks each WHEN condition from top to bottom. When it finds a match, it returns the THEN value. The END keyword closes the CASE expression.

Query 17 - CASE - Department Names

```
SELECT employee_id, first_name, last_name, department_id,
CASE department_id
    WHEN 10 THEN 'Administration'
    WHEN 20 THEN 'Marketing'
    WHEN 30 THEN 'Purchasing'
    WHEN 40 THEN 'Human Resources'
    WHEN 50 THEN 'Shipping'
    WHEN 60 THEN 'IT'
    WHEN 70 THEN 'Public Relations'
    WHEN 80 THEN 'Sales'
    WHEN 90 THEN 'Executive'
```

```

    WHEN 100 THEN 'Finance'
    WHEN 110 THEN 'Accounting'
    ELSE 'Other'
  END AS "Department Name"
FROM employees
ORDER BY department_id;

```

What this query does: This query changes department_id numbers into readable department names. This form of CASE (called simple CASE) compares department_id with each WHEN value. The ELSE clause handles any department_id that is not in the list. This is very useful because users can see 'Sales' instead of just the number 80.

Query 18 - CASE - Bonus Calculation

```

SELECT first_name, last_name, salary,
CASE
  WHEN salary > 15000 THEN salary * 0.20
  WHEN salary > 10000 THEN salary * 0.15
  WHEN salary > 5000 THEN salary * 0.10
  ELSE salary * 0.05
END AS "Bonus Amount"
FROM employees
ORDER BY salary DESC;

```

What this query does: This query calculates a bonus for each employee based on their salary. Higher salaries get a higher bonus percentage. Employees with salary greater than 15000 get 20% bonus. The CASE expression lets you apply different rules to different rows.

Section 7: Character Functions

Character functions change or get information about text data. These functions are very useful when you need to clean data, search in text, or change the format of text. Oracle has many character functions. Here we practice the most common ones.

Query 19 - UPPER and LOWER

```

SELECT UPPER(first_name) AS "Upper Name",
       LOWER(first_name) AS "Lower Name",
       first_name AS "Original Name"
FROM employees
WHERE department_id = 10;

```

What this query does: UPPER() changes all letters to uppercase. LOWER() changes all letters to lowercase. This query shows both versions side by side with the original name. These functions are useful when you want to standardize text data.

Query 20 - CONCAT

```
SELECT CONCAT(first_name, ' ') || last_name AS "Full Name"
FROM employees
WHERE department_id = 20;
```

What this query does: CONCAT() joins two values together. It takes exactly two arguments. In this query, CONCAT(first_name, ' ') joins the first name with a space. Then || joins the result with last_name. Note that CONCAT is different from || because CONCAT takes only two arguments, but || can join many values in a chain.

Query 21 - SUBSTR

```
SELECT first_name,
       SUBSTR(first_name, 1, 3) AS "First 3 Letters",
       SUBSTR(last_name, -3) AS "Last 3 Letters"
FROM employees
WHERE department_id = 30;
```

What this query does: SUBSTR() gets a part of a text string. SUBSTR(string, start, length) gets 'length' characters from the 'start' position. SUBSTR(first_name, 1, 3) gets the first 3 characters starting from position 1. SUBSTR(last_name, -3) gets the last 3 characters (negative numbers count from the end).

Query 22 - INSTR

```
SELECT first_name, last_name,
       INSTR(last_name, 'a') AS "Position of a"
FROM employees
WHERE INSTR(last_name, 'a') > 0
ORDER BY "Position of a";
```

What this query does: INSTR() finds the position of a character or string inside another string. INSTR(last_name, 'a') returns the position number where the letter 'a' first appears in last_name. If 'a' is not found, it returns 0. The WHERE clause filters to show only employees whose last name contains 'a'.

Query 23 - TRIM

```
SELECT TRIM('S' FROM first_name) AS "Trimmed Name",
       first_name AS "Original Name"
FROM employees
WHERE first_name LIKE 'S%';
```

What this query does: TRIM() removes a specific character from the beginning or end of a string. TRIM('S' FROM first_name) removes the letter S from the start of first_name. So 'Steven' becomes 'teven' and 'Shelley' becomes 'helley'. If you use TRIM without specifying a character, it removes spaces from both sides by default.

Section 8: Numeric Functions

Numeric functions work with numbers. They can round numbers, find the highest or lowest integer, and do mathematical calculations. These functions are very important for data analysis because you

often need to work with numbers in different ways.

Query 24 - ROUND

```
SELECT salary,  
       ROUND(salary / 12, 2) AS "Monthly Salary",  
       ROUND(salary / 12, 0) AS "Monthly (Rounded)"  
FROM employees  
WHERE department_id = 50;
```

What this query does: ROUND(number, decimal_places) rounds a number to the given number of decimal places. ROUND(salary / 12, 2) divides salary by 12 (to get monthly salary) and keeps 2 decimal places. ROUND(salary / 12, 0) rounds to the nearest whole number. If you do not specify decimal places, ROUND rounds to 0 decimal places by default.

Query 25 - CEIL and FLOOR

```
SELECT salary,  
       CEIL(salary / 12) AS "Ceil Monthly",  
       FLOOR(salary / 12) AS "Floor Monthly"  
FROM employees  
WHERE department_id = 80;
```

What this query does: CEIL() returns the smallest integer that is greater than or equal to the number (rounds up). FLOOR() returns the largest integer that is less than or equal to the number (rounds down). For example, if salary / 12 = 1250.33, CEIL gives 1251 and FLOOR gives 1250. These functions always return whole numbers (no decimals).

Query 26 - POWER

```
SELECT employee_id,  
       salary,  
       POWER(salary, 2) AS "Salary Squared"  
FROM employees  
WHERE department_id = 10;
```

What this query does: POWER(base, exponent) raises the base number to the power of the exponent. POWER(salary, 2) calculates salary multiplied by salary (salary squared). For example, if salary is 4400, then $4400 * 4400 = 19,360,000$. This function is useful for mathematical and statistical calculations.

Section 9: Date Functions

Date functions help you work with date values. You can get parts of a date like year, month, or day. These functions are very important in business because many reports need to group or filter data by date.

Query 27 - EXTRACT - Year, Month, Day

```
SELECT first_name, last_name, hire_date,  
       EXTRACT(YEAR FROM hire_date) AS "Year",
```

```
        EXTRACT(MONTH FROM hire_date) AS "Month",
        EXTRACT(DAY FROM hire_date) AS "Day"
FROM employees
WHERE department_id = 90;
```

What this query does: EXTRACT() gets a specific part of a date value. EXTRACT(YEAR FROM hire_date) returns only the year. EXTRACT(MONTH FROM hire_date) returns only the month number (1-12). EXTRACT(DAY FROM hire_date) returns only the day of the month (1-31). This is very useful when you want to analyze data by year or month.

Query 28 - EXTRACT - Filter by Year

```
SELECT first_name, last_name, hire_date
FROM employees
WHERE EXTRACT(YEAR FROM hire_date) = 2005
ORDER BY hire_date;
```

What this query does: This query finds employees who were hired in the year 2005. EXTRACT(YEAR FROM hire_date) gets the year from the hire_date column. Then we check if it equals 2005. This is a common pattern for filtering data by a specific year.

Section 10: Conversion Functions

Conversion functions change data from one type to another type. TO_CHAR converts numbers or dates to text. TO_NUMBER converts text to numbers. TO_DATE converts text to dates. These functions are very important because sometimes Oracle needs data in a specific type to do a calculation or comparison.

Query 29 - TO_CHAR - Number Format

```
SELECT employee_id, salary,
       TO_CHAR(salary, '$99,999.99') AS "Formatted Salary",
       TO_CHAR(salary, 'L99,999.99') AS "Local Salary"
FROM employees
WHERE department_id = 80;
```

What this query does: TO_CHAR(number, format) converts a number to text with a specific format. The format '\$99,999.99' shows the number with a dollar sign, comma separators, and 2 decimal places. For example, salary 14000 becomes '\$14,000.00'. The 'L' format uses the local currency symbol. The number 9 in the format means one digit position.

Query 30 - TO_CHAR - Date Format

```
SELECT first_name, last_name, hire_date,
       TO_CHAR(hire_date, 'DD/MM/YYYY') AS "Hire Date (UK)",
       TO_CHAR(hire_date, 'Month DD, YYYY') AS "Hire Date (Full)"
FROM employees
WHERE department_id = 10;
```

What this query does: TO_CHAR(date, format) converts a date to text in a specific date format. 'DD/MM/YYYY' shows the date as day/month/year (like 17/09/2003). 'Month DD, YYYY' shows the full month name (like 'September 17, 2003'). This is useful when you need dates in a specific format for reports.

Query 31 - TO_DATE

```
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date >= TO_DATE('01-JAN-2005', 'DD-MON-YYYY')
ORDER BY hire_date;
```

What this query does: TO_DATE(text, format) converts a text string into a date value. The format 'DD-MON-YYYY' tells Oracle how to read the text. This query finds employees hired on or after January 1, 2005. TO_DATE is important when you need to compare dates entered as text with date columns in the database.

Query 32 - TO_NUMBER

```
SELECT employee_id, salary,
       salary + TO_NUMBER('500.50') AS "Salary Plus Bonus"
FROM employees
WHERE department_id = 10;
```

What this query does: TO_NUMBER(text) converts a text string to a number. In this query, '500.50' is text, but we want to add it to salary (which is a number). TO_NUMBER('500.50') changes '500.50' from text to the number 500.50. Then we can add it to salary. Without TO_NUMBER, Oracle would give an error because you cannot add text to a number.

Section 11: Combined Queries (Practice)

These queries use more than one topic from this lesson. In real work, you often need to combine different functions and operators in one query. Practice these to improve your SQL skills.

Query 33 - CASE + TO_CHAR + Character Functions

```
SELECT UPPER(first_name) || ' ' || UPPER(last_name) AS "Employee",
       TO_CHAR(salary, '$99,999') AS "Salary",
       CASE
         WHEN salary >= 10000 THEN 'Senior'
         WHEN salary >= 5000 THEN 'Mid Level'
         ELSE 'Junior'
       END AS "Level"
FROM employees
WHERE department_id IN (80, 90)
ORDER BY salary DESC;
```

What this query does: This query combines multiple topics: UPPER() to make names uppercase, || for concatenation, TO_CHAR() to format salary with a dollar sign, CASE to create salary levels, and IN to

filter departments. The result is a clean report with employee names, formatted salaries, and levels.

Query 34 - EXTRACT + CASE + LIKE

```
SELECT first_name, last_name, hire_date,  
       EXTRACT(YEAR FROM hire_date) AS "Hire Year",  
       CASE  
         WHEN EXTRACT(YEAR FROM hire_date) < 2003 THEN 'Old Timer'  
         WHEN EXTRACT(YEAR FROM hire_date) <= 2005 THEN 'Mid Timer'  
         ELSE 'New Comer'  
       END AS "Tenure"  
FROM employees  
WHERE last_name LIKE '%son%'  
ORDER BY hire_date;
```

What this query does: This query finds employees whose last name ends with 'son' using LIKE. It uses EXTRACT to get the hire year and CASE to group employees by how long they have worked. Employees hired before 2003 are 'Old Timer', between 2003-2005 are 'Mid Timer', and after 2005 are 'New Comer'.

Query 35 - NULL + CASE + ROUND

```
SELECT first_name, last_name, salary, commission_pct,  
       CASE  
         WHEN commission_pct IS NULL THEN salary  
         ELSE ROUND(salary + salary * commission_pct, 2)  
       END AS "Total Pay"  
FROM employees  
WHERE department_id = 80  
ORDER BY "Total Pay" DESC;
```

What this query does: This query calculates total pay for employees in the Sales department. If commission_pct is NULL (no commission), the total pay is just the salary. If commission_pct has a value, total pay = salary + salary * commission_pct. ROUND() makes sure the result has 2 decimal places.

Section 12: Quick Reference - All Functions

This table gives you a quick summary of all the topics in this lesson. Use this table to review and remember what each operator and function does.

Operator / Function	Purpose	Example
IN	Check if value is in a list	WHERE id IN (10, 20)
NOT IN	Check if value is NOT in a list	WHERE id NOT IN (10, 20)
LIKE	Search for a pattern	WHERE name LIKE 'S%'
NOT LIKE	Exclude a pattern	WHERE name NOT LIKE 'S%'

IS NULL	Find empty values	WHERE mgr IS NULL
IS NOT NULL	Find non-empty values	WHERE mgr IS NOT NULL
	Join text together	fname ' ' lname
CASE	Conditional logic	CASE WHEN ... THEN ... END
UPPER	Convert to uppercase	UPPER(first_name)
LOWER	Convert to lowercase	LOWER(first_name)
TRIM	Remove characters from sides	TRIM('S' FROM name)
SUBSTR	Get part of a string	SUBSTR(name, 1, 3)
INSTR	Find position of character	INSTR(name, 'a')
CONCAT	Join two strings	CONCAT(a, b)
ROUND	Round to decimal places	ROUND(salary, 2)
CEIL	Round up to integer	CEIL(salary / 12)
FLOOR	Round down to integer	FLOOR(salary / 12)
POWER	Raise to a power	POWER(num, 2)
EXTRACT	Get part of a date	EXTRACT(YEAR FROM date)
TO_CHAR	Convert number/date to text	TO_CHAR(salary, '\$99,999')
TO_DATE	Convert text to date	TO_DATE('01-JAN-2024')
TO_NUMBER	Convert text to number	TO_NUMBER('500.50')

Important: Single Quotes vs Double Quotes in Oracle

In Oracle SQL, single quotes and double quotes have different meanings. Understanding this difference will help you avoid errors in your queries.

Symbol	Name	When to Use	Example
' '	Single Quotes	For string values (text data)	WHERE name = 'Steven'
" "	Double Quotes	For column aliases with spaces	AS "Full Name"